UniSENAI

Trilha de Banco de Dados

Módulo 09: Funções e Procedimentos

Instruções para a melhor prática de Estudo

1. **Leia atentamente todo o conteúdo:** Antes de iniciar qualquer atividade, faça uma leitura detalhada do material fornecido na trilha, compreendendo os conceitos e os exemplos apresentados.

2. **Não se limite ao material da trilha:** Utilize o material da trilha como base, mas busque outros materiais de apoio, como livros, artigos acadêmicos, vídeos, e blogs especializados. Isso enriquecerá o entendimento sobre o tema.

3. **Explore a literatura:** Consulte livros e publicações reconhecidas na área, buscando expandir seu conhecimento além do que foi apresentado. A literatura acadêmica oferece uma base sólida para a compreensão de temas complexos.

4. **Realize todas as atividades propostas:** Conclua cada uma das atividades práticas e teóricas, garantindo que você esteja aplicando o conhecimento adquirido de maneira ativa.

5. **Evite o uso de Inteligência Artificial para resolução de atividades:** Utilize suas próprias habilidades e conhecimentos para resolver os exercícios. O aprendizado vem do esforço e da prática.

6. **Participe de debates:** Discuta os conteúdos estudados com professores, colegas e profissionais da área. O debate enriquece o entendimento e permite a troca de diferentes pontos de vista.

7. **Pratique regularmente:** Não deixe as atividades para a última hora. Pratique diariamente e revise o conteúdo com frequência para consolidar o aprendizado.

8. **Peça feedback:** Solicite o retorno dos professores sobre suas atividades e participe de discussões sobre os erros e acertos, utilizando o feedback para aprimorar suas habilidades.

Essas instruções são fundamentais para garantir um aprendizado profundo e eficaz ao longo das trilhas.

Módulo 9: Funções e Procedimentos Armazenados

No MySQL, **funções** e **procedimentos armazenados** são recursos avançados que permitem encapsular lógica, automatizar tarefas repetitivas e aplicar regras de negócio diretamente no banco de dados. Seu uso promove **reutilização de código**, **padronização** e, em muitos casos, **melhor desempenho**.

Este módulo aborda:

- Funções nativas do MySQL
- Procedimentos armazenados (Stored Procedures)
- Funções definidas pelo usuário (User-Defined Functions – UDFs)

1. Funções no MySQL

Funções no MySQL são usadas para realizar operações específicas, como manipulação de strings, cálculos matemáticos e manipulação de datas e horas.

1.1 Funções de Manipulação de Strings

CONCAT: Concatena (junta) strings.

```
SELECT CONCAT('Olá, ', 'Mundo!') AS saudacao;
-- Resultado: Olá, Mundo!
```

SUBSTRING: Extrai parte de uma string.

```
SELECT SUBSTRING('MySQL é poderoso', 7, 2) AS extrato;
-- Resultado: é
```

LOWER e **UPPER**: Converte strings para letras minúsculas ou maiúsculas.

```
SELECT LOWER('MySQL') AS minusculas, UPPER('mysql') AS maiusculas;
-- Resultado: minusculas: mysql | maiusculas: MYSQL
```

Aplicabilidade

- Gerar identificadores únicos:

```
SELECT CONCAT(id_cliente, '-', LOWER(nome)) AS identificador FROM clientes;
```

1.2 Funções de Data e Hora

NOW: Retorna a data e hora atuais.

```
SELECT NOW() AS data_hora_atual;
```

CURDATE: Retorna a data atual.

```
SELECT CURDATE() AS data_atual;
```

DATE_FORMAT: Formata uma data para um formato específico.

```
SELECT DATE_FORMAT(NOW(), '%d/%m/%Y %H:%i:%s') AS data_formatada;
-- Resultado: 31/12/2024 14:30:45
```

DATEDIFF: Calcula a diferença em dias entre duas datas.

```
SELECT DATEDIFF('2024-12-31', '2024-12-25') AS diferenca_dias;
-- Resultado: 6
```

Aplicabilidade

- Relatórios com intervalos de datas:

```
SELECT id_pedido, DATEDIFF(CURDATE(), data_pedido) AS dias_atraso
FROM pedidos
WHERE status = 'atrasado';
```

1.3 Funções Matemáticas

ROUND: Arredonda um número para o número de casas decimais especificado.

```
SELECT ROUND(123.456, 2) AS arredondado;
-- Resultado: 123.46
```

FLOOR e **CEIL**: Arredondam para baixo ou para cima, respectivamente.

```
SELECT FLOOR(123.456) AS para_baixo, CEIL(123.456) AS para_cima;
-- Resultado: para_baixo: 123 | para_cima: 124
```

ABS: Retorna o valor absoluto.

```
SELECT ABS(-50) AS absoluto;
-- Resultado: 50
```

Aplicabilidade

- Cálculo de totais e arredondamentos para relatórios financeiros.

---

2. Procedimentos Armazenados (Stored Procedures)

2.1 O que são Procedimentos Armazenados

Um **procedimento armazenado** é um conjunto de instruções SQL que pode ser executado como uma unidade, permitindo a execução de tarefas complexas de forma repetitiva e eficiente.

- **Vantagens**:
  - Redução de redundância de código.
  - Melhoria na performance, pois o código é armazenado no servidor.
  - Maior segurança com controle de acesso.

2.2 Criação de Procedimentos

Sintaxe Geral

```
CREATE PROCEDURE nome_procedimento(parametros)
BEGIN
    corpo_do_procedimento;
END;
```

Exemplo Prático

Criar um procedimento que lista todos os pedidos de um cliente específico:

```
CREATE PROCEDURE listar_pedidos_cliente(IN cliente_id INT)
BEGIN
    SELECT * FROM pedidos WHERE id_cliente = cliente_id;
END;
```

1. `IN`: Parâmetro de entrada.

Execução de Procedimentos Armazenados

Os procedimentos armazenados são executados com o comando `CALL`.

Exemplo: Executar o procedimento criado:

```
CALL listar_pedidos_cliente(1);
```

---

3. Funções Definidas pelo Usuário (UDFs)

3.1 O que são UDFs

As **User-Defined Functions (UDFs)** permitem criar funções personalizadas que:

- Retornam **um único valor**
- Podem ser usadas em `SELECT`, `WHERE`, `ORDER BY`

⚠️ Diferente de procedures, **funções não devem alterar dados**.

3.2 Criação de Funções Customizadas

Sintaxe Geral

```
CREATE FUNCTION nome_funcao(parametros)
RETURNS tipo_de_retorno
DETERMINISTIC
BEGIN
    corpo_da_funcao;
    RETURN valor;
END;
```

Exemplo Prático

Criar uma função que calcula o desconto aplicado a um valor:

```
CREATE FUNCTION calcular_desconto(valor DECIMAL(10,2), desconto DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    RETURN valor - (valor * desconto / 100);
END;
```

Executar a função:

```
SELECT calcular_desconto(200, 10) AS valor_com_desconto;
-- Resultado: 180.00
```

Aplicabilidade

- **Funções no MySQL**:
  - Manipulação de strings em relatórios, como transformar nomes em formatos específicos.
  - Cálculos matemáticos para criar análises financeiras detalhadas.
  - Operações com datas, como calcular vencimentos e prazos.
- **Procedimentos Armazenados**:
  - Tarefas rotineiras, como gerar relatórios de vendas ou atualizar estoques.
  - Processos complexos, como validações e cálculos automáticos durante a inserção de dados.
- **Funções Customizadas**:
  - Criação de cálculos específicos reutilizáveis em consultas, como margens de lucro ou taxas personalizadas.

Com o uso de funções e procedimentos armazenados, é possível criar sistemas mais eficientes, organizados e fáceis de manter. Isso melhora a performance e facilita a implementação de regras de negócio diretamente no banco de dados.

---

4. Diferença entre FUNCTION e PROCEDURE

Aspecto	FUNCTION	PROCEDURE
Retorno	Obrigatório (1 valor)	Opcional
Uso em SELECT	Sim	Não
Altera dados	Não recomendado	Sim
Execução	Dentro de consultas	CALL

Parte 1: Questões Práticas

- **Manipulação de Strings**:
  - Crie uma consulta que utilize a função `CONCAT` para gerar um identificador único combinando o nome e o ID de um cliente.
- **Função `SUBSTRING`**:
  - Extraia os três primeiros caracteres do nome de um produto na tabela `produtos`.
- **Uso de `LOWER` e `UPPER`**:
  - Converta todos os nomes de clientes para letras maiúsculas em uma consulta SQL.
- **Funções de Data e Hora**:
  - Escreva uma consulta que utilize `NOW` para exibir a data e hora atuais, formatando a saída para o formato `DD/MM/AAAA HH:MM:SS` com `DATE_FORMAT`.
- **Diferença de Datas**:
  - Calcule o número de dias entre a data de hoje e a data de um pedido na tabela `pedidos` usando a função `DATEDIFF`.
- **Arredondamento**:
  - Arredonde o valor de um campo de preço para duas casas decimais usando a função `ROUND`.
- **Arredondar para Cima/Para Baixo**:
  - Use `FLOOR` para arredondar para baixo e `CEIL` para arredondar para cima o valor de um campo de total de vendas.
- **Criação de Procedimentos Armazenados**:
  - Crie um procedimento armazenado que insira um novo pedido na tabela `pedidos`, recebendo como entrada o ID do cliente e a data do pedido.
- **Execução de Procedimentos Armazenados**:
  - Execute o procedimento criado no exercício anterior para adicionar um pedido com seus parâmetros.
- **Criação de Funções Definidas pelo Usuário (UDFs)**:
  - Crie uma função que calcule o valor total de um pedido aplicando um desconto percentual e use-a em uma consulta.

Parte 2: Questões Teóricas

- **Definição de Funções**:
  - O que são funções no MySQL e quais os tipos mais comuns?
- **Função `CONCAT`**:
  - Explique como a função `CONCAT` pode ser usada para manipular dados em relatórios.
- **Funções de Data e Hora**:
  - Qual a diferença entre `NOW` e `CURDATE`? Cite exemplos de suas aplicações.
- **Arredondamento**:
  - Qual é a diferença entre as funções `ROUND`, `FLOOR` e `CEIL`? Em que situações cada uma é mais adequada?

- **Vantagens dos Procedimentos Armazenados**:
  - Liste três vantagens de usar procedimentos armazenados em um banco de dados MySQL.
- **Parâmetros de Procedimentos**:
  - Qual a diferença entre os tipos de parâmetros `IN`, `OUT` e `INOUT` em procedimentos armazenados?
- **Execução de Procedimentos**:
  - Qual comando é usado para executar um procedimento armazenado e como ele funciona?
- **Funções Definidas pelo Usuário (UDFs)**:
  - O que são UDFs e como elas diferem de procedimentos armazenados?
- **Aplicações de Funções Personalizadas**:
  - Explique uma situação prática onde a criação de uma função definida pelo usuário seria mais vantajosa do que o uso de uma função padrão.
- **Funções e Procedimentos**:
  - Compare as funções e os procedimentos armazenados em termos de retorno de valores e uso em consultas.